Pretrained Models

From ImageNet to foundation models

Machine Learning with Python — On-site session (Oct 23)

EUGLOH — *Problem Solving Using Open-Source Languages; R and Python*

University of Novi Sad

Today's session

- **Why** use someone else's trained network?
- **Feature extraction** — a pretrained model as a powerful feature map
- **Linear probing vs fine-tuning** — the two ways to reuse it
- **The course dataset** — how we will process it (details to come)
- **Foundation models** — ViT, CLIP, and zero-/few-shot learning
- **Putting it together** with PCA/t-SNE and the models you already know

On-site session — the second of two today.

Part 1 — Why pretrained models?

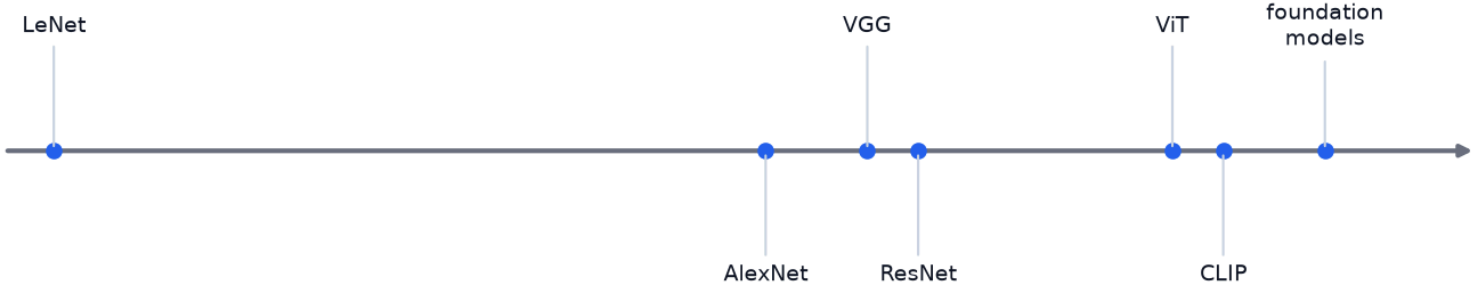
Training a good image model **from scratch** needs a huge amount of data and compute: millions of labelled images, many GPUs, and days of training. You probably have none of that.

But the features such a model learned are **general**: edges, textures, shapes, object parts. They are useful far beyond the exact task the model was trained on.

- **Free, strong features.** Take a network trained on a large dataset and reuse its internal representation.
- **Works with little data.** A simple classifier on top of good features often beats a complex model on raw pixels.
- **The modern default.** Almost nobody trains vision models from scratch — they **transfer**.

In plain words: **stand on the shoulders of a model that already learned to see.**

A short history of pretrained vision models



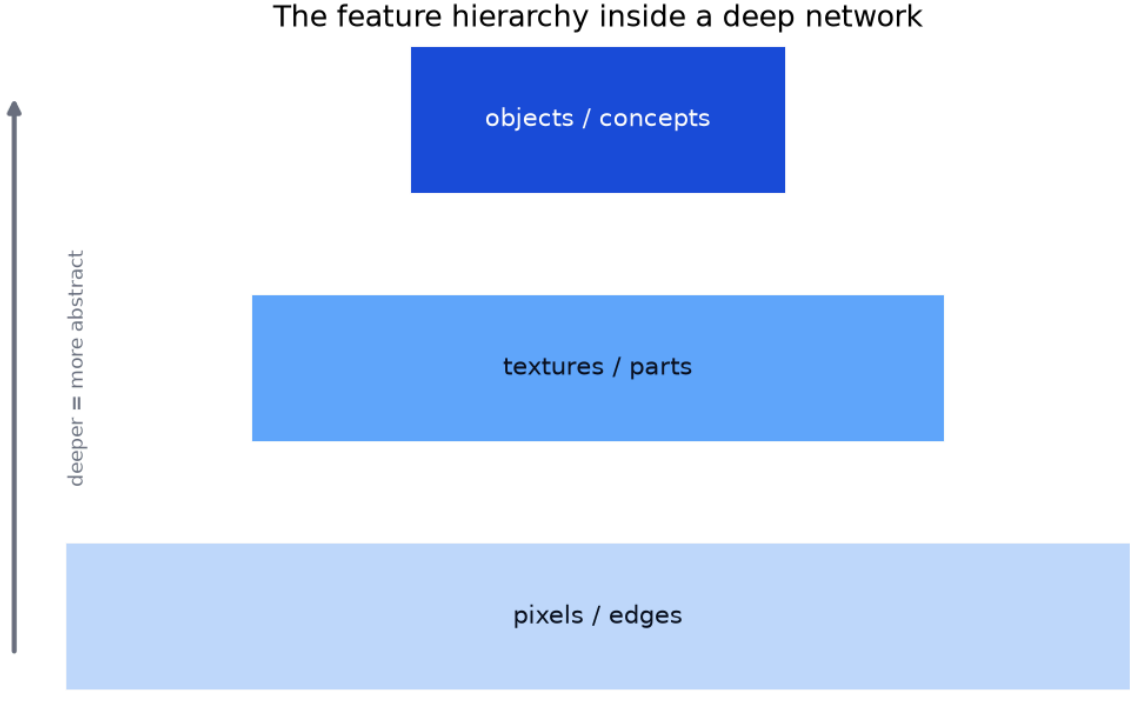
What do the layers learn?

A deep network builds its representation **hierarchically**:

- **Early layers** see colours and **edges** — simple, local patterns.
- **Middle layers** combine them into **textures and parts** (corners, wheels, eyes).
- **Late layers** encode **objects and concepts** — "a dog", "a car".

Because the late layers are already about *things*, their activations are an excellent description of an image — one that transfers to new tasks.

ImageNet (≈1.2 million labelled images, 1000 classes) is the dataset that made this practical: a model that is good on ImageNet has learned a broadly useful vocabulary of visual features.


The feature hierarchy inside a deep network
objects / concepts
textures / parts
pixels / edges
deeper = more abstract

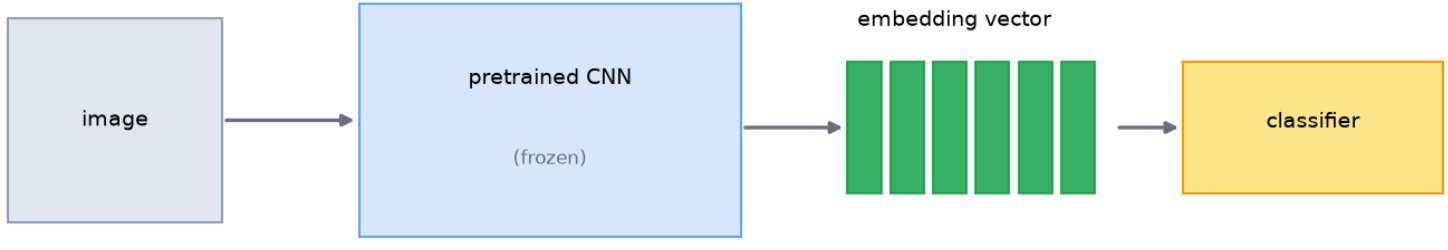
Feature extraction — the key idea

Chop the **classifier head** off a pretrained network and keep the layer just before it. For every image, that layer produces a **feature vector** (an **embedding**) — perhaps 512 or 1024 numbers that summarise the image.

- **Similar images land near each other** in embedding space.
- The embedding was learned on a huge dataset, so it already captures shape, texture and semantics.
- You can now use **any** classifier you have learned — logistic regression, a random forest, a small MLP — on these embeddings instead of on raw pixels.

In plain words: **turn each image into a compact, meaningful vector, then apply what you already know.**

Feature extraction: reuse the network as a feature map

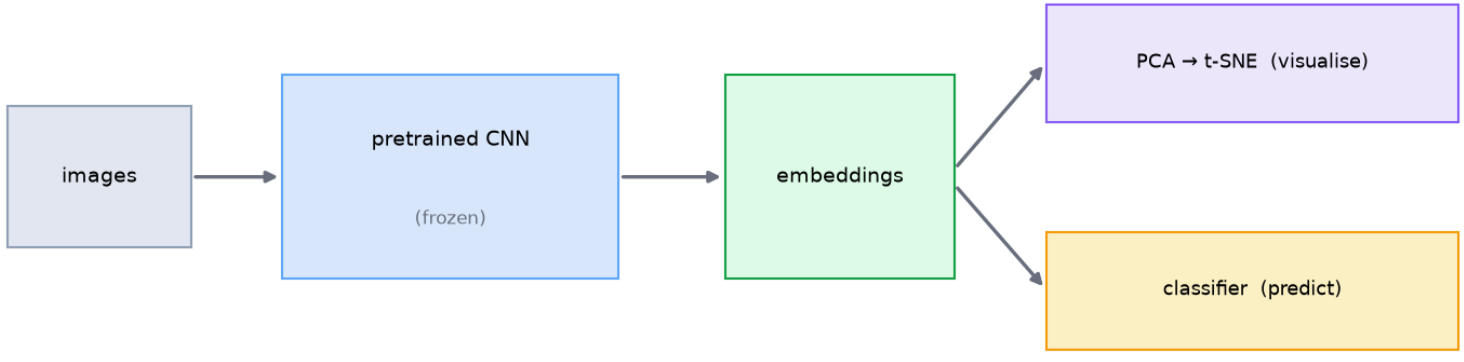


Two ways to reuse a pretrained model

	Linear probing (feature extraction)	**Fine-tuning**
what changes	only the new classifier	the whole network
data needed	little	more
compute	seconds–minutes	much more
risk	underfits if features are a poor match	overfits / forgets if data is small
when	your default first try	features are a poor match and you have data

Start with **linear probing** (just logistic regression on the embeddings) — it is cheap, robust, and surprisingly strong. Reach for **fine-tuning** only when it is clearly not enough.

The pipeline: pretrained features, then visualise or predict



Part 2 — The course dataset

> ***Placeholder — details to be confirmed.*** *This section will introduce the dataset we use in the hands-on part and walk through the pipeline on it.*

The plan for the hands-on part:

1. **Look** at the data — what are the inputs, how many classes and samples?
2. **Prepare** it — resize, normalise, and split train / validation / test.
3. **Extract** a pretrained embedding for every sample (frozen network).
4. **Visualise** the embeddings with **PCA / t-SNE** (Lecture 1).
5. **Classify** them with logistic regression / random forest and compare against a baseline.

Part 3 — Foundation models

Pretrained models have grown from **task-specific** (an ImageNet classifier) to **general-purpose** — models whose representations work across many tasks and even across modalities.

- **Vision Transformer (ViT)** — split the image into patches and apply the same attention mechanism as language models.
- **CLIP** — trained on 400M *image–text* pairs, it places images and captions in **one shared space**, so you can match an image to text.
- **Self-supervised** models (e.g. **DINO**) — learn from images *without labels*; the embeddings are excellent and very transferable.
- **Segment Anything (SAM)** — promptable segmentation; a foundation model for masks.
- **Large language models** — the text-side analog: pretrain once, adapt to almost anything.

The pattern is the same as before — **pretrain on a lot, reuse broadly** — just at a much larger scale.

Zero-shot and few-shot

Zero-shot — with CLIP, no training examples are needed at all. Write the class names as text prompts, embed them, and assign each image the class whose prompt is closest:

> *"a photo of a **cat**"* · *"a photo of a **dog**"* · *"a photo of a **bird**"*

Because images and text live in the same space, this often works out of the box.

Few-shot — a handful of labelled examples per class, then a **linear probe** on the frozen embeddings. A few images often go a long way.

This is the practical payoff of foundation models: **strong results with little or no task-specific training data.**

What to watch out for

- **Domain gap.** Features learned on everyday photos may transfer poorly to very different data (medical scans, satellite imagery).
- **Compute & memory.** Even *frozen* models can be slow on large data; embeddings must be computed once and stored.
- **Licensing & usage.** Pretrained weights and data come with terms — check before you deploy.
- **Bias.** The pretraining data is not neutral; its biases end up in the features.
- **Evaluate properly.** Always compare against a **baseline** (raw pixels, or a simple model) on a **held-out test set** — a fancy embedding is not automatically better.

Summary

- A **pretrained model** gives you strong, general **features** for free — almost nobody trains vision models from scratch.
- **Feature extraction** turns each input into an **embedding** you can feed to any classifier; **fine-tuning** updates the whole network when you have enough data.
- **Foundation models** (ViT, CLIP, DINO, SAM) push this further, enabling **zero- and few-shot** learning.
- Combined with **PCA / t-SNE** for visualisation and the **supervised models** from earlier, pretrained features let us explore and classify real image data with very little training.

Where to go next

- **Hands-on:** apply the full pipeline — pretrained embeddings → PCA / t-SNE → classifier — to the **course dataset** (details to come).
- Use the supervised models from earlier in the course (logistic regression, forests, a small MLP) on the **embeddings**, and compare with the same models on **raw pixels**.